```
# Step 0: Install required packages
!pip install -q nltk spacy scikit-learn

# Step 1: Download necessary NLTK and spaCy data
import nltk
nltk.download('stopwords', force=True)
nltk.download('wordnet', force=True)
!python -m spacy download en_core_web_sm

# Load spaCy model
import spacy
nlp = spacy.load("en_core_web_sm")

# Step 2: Import libraries
import pandas as pd
from nltk.tokenize import RegexpTokenizer
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report

# Step 3: Text Collection
text = """
Natural Language Processing (NLP) is a subfield of artificial
intelligence that deals with the interaction between computers and
humans through natural language. Applications of NLP include
sentiment analysis, machine translation, chatbots, and more.
"""
print("Original Text:\n", text)
```

```
# Step 4: Preprocessing function using RegexpTokenizer (fix for
punkt_tab error)
def preprocess(text):
    text = text.lower()
    tokenizer = RegexpTokenizer(r'\w+')
    tokens = tokenizer.tokenize(text)
    stop_words = set(stopwords.words('english'))
    tokens = [w for w in tokens if w not in stop_words]
    lemmatizer = WordNetLemmatizer()
    tokens = [lemmatizer.lemmatize(w) for w in tokens]
    return ' '.join(tokens)
clean_text = preprocess(text)
print("\nPreprocessed Text:\n", clean_text)

# Step 5: Text Representation with TF-IDF
vectorizer = TfidfVectorizer()
X_tfidf = vectorizer.fit_transform([clean_text])
print("\nTF-IDF Features:\n", vectorizer.get_feature_names_out())

# Step 6: Feature Engineering using spaCy
doc = nlp(text)
print("\nPOS Tags:")
for token in doc:
    print(f"{token.text} --> {token.pos_}")
print("\nNamed Entities:")
for ent in doc.ents:
    print(f"{ent.text} --> {ent.label_}")
print("\nDependency Parsing:")
for token in doc:
    print(f"{token.text} <--{token.dep_}-- {token.head.text}")

# Step 7: Model Training with larger sample sentiment data
sample_data = [
    ("I love this product, it’s amazing!", "positive"),
    ("This is the worst experience I've ever had.", "negative"),
```

```
    ("The service was okay, not the best.", "neutral"),
    ("Absolutely fantastic! Will use again.", "positive"),
    ("Horrible, never coming back.", "negative"),
    ("Not bad, but could be better.", "neutral"),
    ("Best purchase I have made.", "positive"),
    ("I hate it, very disappointing.", "negative"),
    ("It works well, I am happy.", "positive"),
    ("Mediocre service, nothing special.", "neutral"),
    ("Terrible, I want a refund.", "negative"),
    ("Pretty good overall.", "positive"),
    ("Average experience, neither good nor bad.", "neutral"),
    ("I do not recommend this.", "negative"),
    ("Exceeded my expectations!", "positive"),
]
df = pd.DataFrame(sample_data, columns=["text", "label"])
df["text_clean"] = df["text"].apply(preprocess)
le = LabelEncoder()
df["label_encoded"] = le.fit_transform(df["label"])
# Stratified train-test split to preserve label distribution
X_train, X_test, y_train, y_test = train_test_split(
    df["text_clean"], df["label_encoded"], test_size=0.3,
random_state=42, stratify=df["label_encoded"]
)
pipeline = Pipeline([
    ('tfidf', TfidfVectorizer()),
    ('clf', MultinomialNB())
])
pipeline.fit(X_train, y_train)

# Step 8: Evaluation with fix (labels specified explicitly)
y_pred = pipeline.predict(X_test)
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred,
labels=le.transform(le.classes_), target_names=le.classes_))
# Show test examples with predictions
```

```
print("\nTest samples and predictions:")
for text_sample, true_label, pred_label in zip(X_test,
le.inverse_transform(y_test), le.inverse_transform(y_pred)):
    print(f"Text: {text_sample}")
    print(f"True: {true_label} | Predicted: {pred_label}")
    print("---")

# Step 9: Deployment demo
sample_input = "The product is excellent and I highly recommend it."
processed_input = preprocess(sample_input)
predicted_label = pipeline.predict([processed_input])[0]
print("\nPredicted Sentiment:",
le.inverse_transform([predicted_label])[0])

# Step 10: Monitoring & maintenance placeholder
print("\nMonitoring and maintenance steps would be implemented in
production.")
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
Collecting en-core-web-sm==3.8.0
  Downloading
https://github.com/explosion/spacy-models/releases/download/en_core_
web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl (12.8 MB)
```



```
12.8/12.8 MB 93.6 MB/s eta 0:00:00
✔ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart
Python in
order to load all the package's dependencies. You can do this by
selecting the
'Restart kernel' or 'Restart runtime' option.
```

Original Text:

Natural Language Processing (NLP) is a subfield of artificial intelligence that deals with the interaction between computers and humans through natural language. Applications of NLP include sentiment analysis, machine translation, chatbots, and more.

Preprocessed Text:
 natural language processing nlp subfield artificial intelligence deal interaction computer human natural language application nlp include sentiment analysis machine translation chatbots

TF-IDF Features:
 ['analysis' 'application' 'artificial' 'chatbots' 'computer' 'deal'
 'human' 'include' 'intelligence' 'interaction' 'language' 'machine'
 'natural' 'nlp' 'processing' 'sentiment' 'subfield' 'translation']

POS Tags:

 --> SPACE
Natural --> PROPN
Language --> PROPN
Processing --> PROPN
(--> PUNCT
NLP --> PROPN
) --> PUNCT
is --> AUX
a --> DET
subfield --> NOUN
of --> ADP
artificial --> ADJ
intelligence --> NOUN
that --> PRON
deals --> VERB
with --> ADP
the --> DET
interaction --> NOUN
between --> ADP
computers --> NOUN
and --> CCONJ
humans --> NOUN

```
through --> ADP
natural --> ADJ
language --> NOUN
. --> PUNCT
Applications --> NOUN
of --> ADP
NLP --> PROPN
include --> VERB
sentiment --> NOUN
analysis --> NOUN
, --> PUNCT
machine --> NOUN
translation --> NOUN
, --> PUNCT
chatbots --> NOUN
, --> PUNCT
and --> CCONJ
more --> ADJ
. --> PUNCT

 --> SPACE

Named Entities:
NLP --> ORG
NLP --> ORG

Dependency Parsing:

 <--dep-- Processing
Natural <--compound-- Language
Language <--compound-- Processing
Processing <--nsubj-- is
( <--punct-- Processing
NLP <--appos-- Processing
) <--punct-- Processing
is <--ROOT-- is
a <--det-- subfield
subfield <--attr-- is
of <--prep-- subfield
artificial <--amod-- intelligence
intelligence <--pobj-- of
```

```
that <--nsubj-- deals
deals <--relcl-- intelligence
with <--prep-- deals
the <--det-- interaction
interaction <--pobj-- with
between <--prep-- interaction
computers <--pobj-- between
and <--cc-- computers
humans <--conj-- computers
through <--prep-- computers
natural <--amod-- language
language <--pobj-- through
. <--punct-- is
Applications <--nsubj-- include
of <--prep-- Applications
NLP <--pobj-- of
include <--ROOT-- include
sentiment <--compound-- analysis
analysis <--dobj-- include
, <--punct-- analysis
machine <--compound-- translation
translation <--conj-- analysis
, <--punct-- translation
chatbots <--conj-- translation
, <--punct-- chatbots
and <--cc-- chatbots
more <--conj-- chatbots
. <--punct-- include

 <--dep-- .
```

Classification Report:

	precision	recall	f1-score	support
negative	0.00	0.00	0.00	2
neutral	0.00	0.00	0.00	1
positive	0.25	0.50	0.33	2
accuracy			0.20	5
macro avg	0.08	0.17	0.11	5

```
weighted avg       0.10      0.20      0.13         5


Test samples and predictions:
Text: absolutely fantastic use
True: positive | Predicted: positive
---
Text: average experience neither good bad
True: neutral | Predicted: positive
---
Text: worst experience ever
True: negative | Predicted: positive
---
Text: best purchase made
True: positive | Predicted: neutral
---
Text: terrible want refund
True: negative | Predicted: positive
---

Predicted Sentiment: positive

Monitoring and maintenance steps would be implemented in production.
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classificat
ion.py:1565: UndefinedMetricWarning: Precision is ill-defined and
being set to 0.0 in labels with no predicted samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classificat
ion.py:1565: UndefinedMetricWarning: Precision is ill-defined and
being set to 0.0 in labels with no predicted samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classificat
ion.py:1565: UndefinedMetricWarning: Precision is ill-defined and
being set to 0.0 in labels with no predicted samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
```